WEB 250 asgn08 Sandbox code flow

Student review handout 2 of 5 | Ungraded practice Goal: Follow one browser request, watch database rows become objects, and explain what comes back to the page.

Use your working asgn08 project. The Sandbox line numbers below match the original asgn08_student_starter.zip. If you added comments or completed an experiment, find the **statement shown**, even if its line number has moved. Keep your working code; the debugger helps you examine it.

1 Start with the first index request

Open your local asgn08/public/index.php in the browser. If you first open the course index at web250/index.php, clicking its asgn08 link starts the asgn08 request described here. A link requests a new page; it does not include that file in the current PHP execution.

In the completed asgn08 project, execution follows this order:

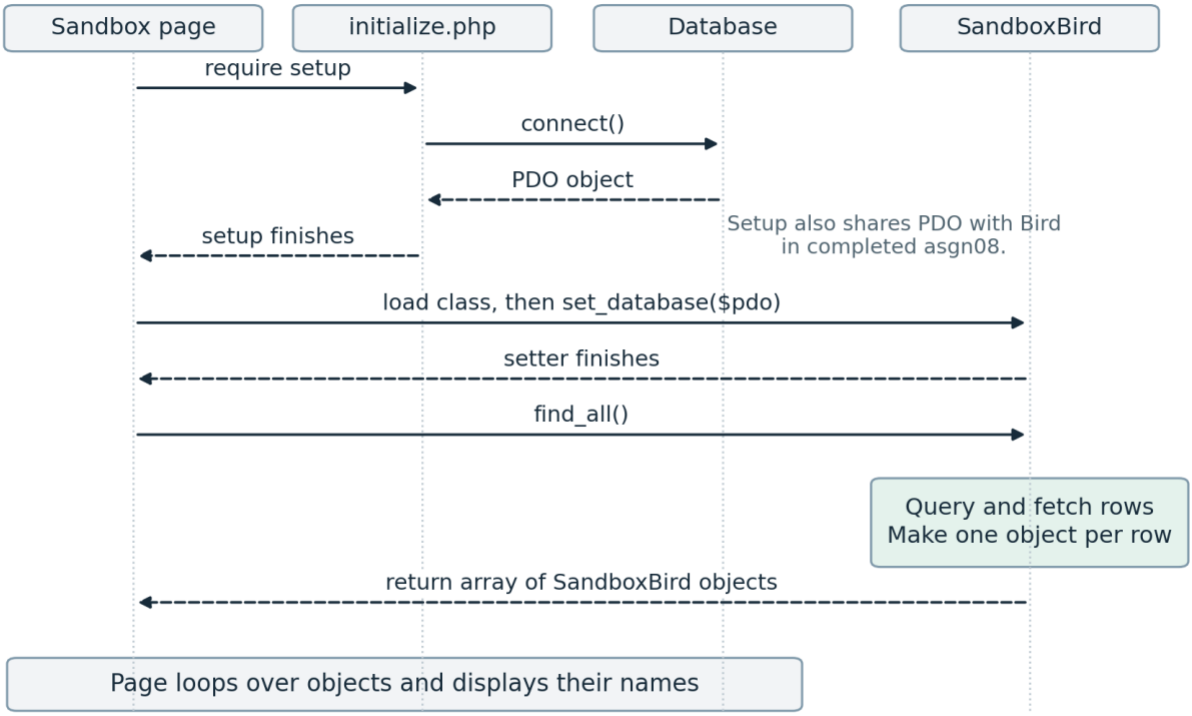
Order	File or method	What happens
1	public/index.php	The server begins executing the PHP file requested by the browser.
2	require_once … '/private/initialize.php'	PHP pauses the index at this statement and executes the included setup file.
3	private/initialize.php	The setup loads the helper, credentials, Database class, and Bird class. Loading class definitions makes their methods available; it does not run every method.
4	$pdo = Database::connect();	PHP enters the method. On this request's first connection, it builds the DSN and calls new PDO(…). The returned PDO object is assigned to $pdo.
5	Bird::set_database($pdo);	The model stores the same connection in its private static property. This does not run a bird query.
6	Return to public/index.php	When initialization finishes, PHP resumes after the require_once statement.
7	Bird::count_all() and the existing summary queries	The index retrieves the bird count and the existing image and image-link counts. Counting birds does not create a collection of Bird objects.
8	The index HTML	PHP inserts the counts into HTML. The browser receives the resulting page with its navigation links.

The earlier asgn07 setup may still connect inside the public page. If that is your current version, the first Database::connect() happens there. The asgn08 instructions move connection sharing into initialize.php so the completed public pages can use Bird.

Clicking a link starts another request

When you click the **Active Record Sandbox** link, the browser requests public/sandbox_active_record.php. **The index does not call or include the Sandbox.** Its earlier request has finished.

The Sandbox starts at its own first PHP statement and loads its setup for this new request. The PDO connection references and the bird objects in the earlier request do not carry over into this one. require_once means once within the current request; it does not mean once for the lifetime of the website.



Method calls during the Sandbox request

2 The complete Sandbox path

Read this table once before stepping through the code. The two loops have different jobs: the first creates objects; the second displays them.

Stop in the code	What it does	Result after it runs
Page, line 5: require_once … '/private/initialize.php';	Loads the shared setup and runs any setup statements in that file.	The connection class and helpers are available; completed asgn08 setup also supplies $pdo.
Page, line 6: require_once for SandboxBird.php	Loads the Sandbox class definition.	PHP knows the properties and methods. It has not called find_all().
Page, line 10: PDO reuse expression	Uses an existing PDO object if setup supplied one; otherwise calls Database::connect().	$pdo is a PDO connection object. Only the selected side of the ternary runs.
Page, line 11: set_database($pdo)	Enters the Sandbox setter and executes class line 20, self::$database = $pdo;.	The Sandbox class can use that same connection.

Stop in the code	What it does	Result after it runs
Page, line 12: find_all()	Enters the Sandbox finder. The assignment to the page's $birds must wait for the method to return.	The result will eventually be an array of SandboxBird objects.
Class, lines 31–32: SQL string, then query($sql)	Requests id and common_name, ordered by common name.	$statement is a PDOStatement object.
Class, line 33: fetchAll(PDO::FETCH_ASSOC)	Fetches the result as associative arrays and assigns it to $rows.	$rows is an array containing one associative array per returned record.
Class, lines 35–41: initialize $birds, then the loop	For each $row, creates a new object, copies the ID and name, and appends the object to the local $birds array.	One populated SandboxBird object per returned row.
Class, line 44: return $birds;	Sends the method's result back to the calling page.	The page's pending assignment receives the returned array.
Page, lines 33 and 35	Counts the returned objects and chooses the empty or nonempty display.	With the original seed data, the count is 8.
Page, lines 39–42: HTML loop	Visits each object, reads its properties, and calls sandbox_h() to escape the name.	The browser receives a list of names and record IDs.

sandbox_h() calls htmlspecialchars() and returns escaped text. It does not change the database record or create another bird object.

What changes shape

Value	Example during the first loop iteration
$row	An associative array with id and common_name keys. Read the name with $row['common_name'].
$bird, just after new self()	A SandboxBird object with the defaults id = 0 and common_name = ''.
$bird, after both assignments	A SandboxBird object with its database ID and common_name = 'American Robin' on the original seed.
$birds, after the first append	An array containing that one object at index 0. Array index 0 is not the bird's database ID.

Read an associative row with $row['common_name'] and an object's property with $bird->common_name.

The assignments from $row into $bird are the row-to-object conversion, also called **hydration**. new self() creates an empty/default object; it does not fetch the record by itself.

3 Notice the two return trips

The setup file returns control to the page after its last statement. Later, find_all() returns an actual value: an array of objects. These are two different uses of the word return. An included file finishing does not mean the entire request has finished.

Inside find_all(), $birds is a local variable. The page also calls its result $birds, but it receives that result only when the method finishes. At the page assignment, first evaluate the right side, SandboxBird::find_all(), then store its result on the left.

The first foreach reads associative arrays and constructs objects. The second foreach, in the HTML, reads those already constructed objects and produces output. Neither printing a property nor calling sandbox_h() fetches another row.

4 Check your explanation

Before using handout 3, point to the code that answers each question.

1. Where does this browser request begin?

2. Which method opens the connection, and which method shares it with the Sandbox class?

3. Where is the first object created? What properties does it have at that moment?

4. What does find_all() give back to the public page?

5. Which loop makes objects, and which loop makes the displayed list?

Code reference: Original asgn08_student_starter.zip, specifically SandboxBird.php and sandbox_active_record.php; asgn08 instructions Sections 2 and 5; original asgn07 connection class and seed SQL. Names/counts assume the original eight species. Added records or completed Sandbox experiments may change them.

PHP references: Static methods and properties, require_once, and PDOStatement fetchAll.